



Rapport de Projet de Fin d'Études

SLAM Multi-Capteurs

MC-SLAM

Matis DUVAL

Année universitaire 2025–2026

Résumé

Ce rapport présente le travail réalisé dans le cadre du [Projet de Fin d'Études \(PFE\)](#) intitulé *SLAM Multi-Capteurs (MC-SLAM)*. L'objectif principal est la prise en main de la plateforme TurtleBot 4 sous [Robot Operating System \(ROS\) 2](#), suivi du développement d'une solution de [Simultaneous Localization and Mapping \(SLAM\)](#) exploitant une caméra [RGB](#) et un [LiDAR](#).

Table des matières

1	Introduction	5
1.1	Contexte	5
1.2	Problématique	5
1.3	Sujet et objectifs	6
2	État de l’art	7
2.1	SLAM visuel (Visual SLAM)	7
2.2	SLAM LiDAR	7
2.3	SLAM multi-capteurs	7
2.4	SLAM thermique (LWIR)	8
2.5	3D Gaussian Splatting	8
3	User Stories	9
4	Spécification des besoins	10
4.1	Besoins fonctionnels	10
4.2	Besoins non fonctionnels	10
4.3	Tests	11
5	Choix logiciels	12
5.1	Système d’exploitation et middleware	12
5.2	Algorithmes de SLAM	12
5.3	Calibration	13
5.4	Évaluation	13
5.5	Développement et versioning	13
6	Architecture	14
6.1	Architecture matérielle	14
6.2	Architecture logicielle ROS 2	14
6.3	Diagramme de cas d’utilisation (UML)	16

7	Réalisation	17
7.1	Mise en place de l'environnement	17
7.2	Screenshots	18
7.3	Algorithmes et méthodes	18
7.3.1	ICP from scratch	18
7.3.2	Calibration extrinsèque caméra/LiDAR	20
8	Résultats et tests	21
8.1	Résultats expérimentaux	21
9	Conclusion	22
9.1	Bilan	22
9.2	Perspectives	22
	Références bibliographiques	23

Table des figures

7.1	Visualisation des cartes générées par les deux pipelines SLAM	18
-----	---	----

Liste des tableaux

3.1	User stories principales	9
4.1	Plan de tests	11
8.1	Comparaison des performances SLAM sur la séquence de test	21

Chapitre 1

Introduction

1.1 Contexte

La robotique mobile autonome connaît un essor considérable, portée par les avancées en intelligence artificielle, en vision par ordinateur et en capteurs embarqués. Dans ce contexte, la capacité d'un robot à se localiser tout en construisant une carte de son environnement, le [Simultaneous Localization and Mapping \(SLAM\)](#) est devenu une brique fondamentale de nombreuses applications: navigation en intérieur, véhicules autonomes, inspection industrielle ou encore robotique de service.

Ce Projet vise à exploiter la plateforme robotique **TurtleBot 4** pour développer et comparer différentes approches de [SLAM](#) multi-capteurs.

1.2 Problématique

Malgré la l'avancé des algorithmes de [SLAM](#) monoculaires ou monocapteurs, plusieurs verrous subsistent:

- **Robustesse aux conditions d'éclairage:** les approches [RGB](#) sont sensibles aux variations lumineuses; les capteurs [LWIR](#) offrent une alternative thermique mais restent peu explorés dans ce contexte.
- **Fusion multi-capteurs :** la combinaison [LiDAR](#) + caméra peut améliorer la précision, mais nécessite une calibration rigoureuse.
- **Représentation dense de l'environnement:** les cartes éparses issues des approches classiques sont limitées pour la navigation fine et la reconstruction 3D photo-réaliste.

1.3 Sujet et objectifs

Les objectifs du projet sont les suivants:

1. **Prise en main** de la plateforme TurtleBot 4 (architecture matérielle, logicielle sous [ROS 2](#), calibration des capteurs).
2. **Développement d'un pipeline MC-SLAM** exploitant la caméra [RGB](#) et le [LiDAR](#) intégré.

Chapitre 2

État de l’art

2.1 SLAM visuel (Visual SLAM)

Le SLAM visuel utilise des caméras comme capteur principal. Parmi les systèmes les plus répandus, ORB-SLAM3 propose une architecture complète supportant les caméras monoculaire, stéréo et RGB-D, avec une gestion de la réinitialisation et des cartes multi-sessions [1]. DSO (*Direct Sparse Odometry*) adopte quant à lui une approche directe fondée sur la minimisation de l’erreur photométrique, ce qui le rend particulièrement adapté aux scènes peu texturées ou d’autres capteurs sont erronées. Enfin, VINS-Mono et VINS-Fusion intègrent la vision et une centrale inertielle (IMU) afin de fournir une odométrie visuo-inertielle précise.

2.2 SLAM LiDAR

Les approches LiDAR offrent une précision métrique supérieure en extérieur et dans les larges espaces. LOAM et sa variante légère LeGO-LOAM reposent sur l’extraction de *features* géométriques des arêtes et plans pour assurer l’odométrie et la cartographie en temps réel. LIO-SAM étend cette approche en fusionnant les données LiDAR et IMU au sein d’une optimisation par graphe de facteurs. À l’opposé, KISS-ICP propose un pipeline minimaliste et robuste basé sur l’ICP adaptatif, particulièrement adapté aux plateformes embarquées [2].

2.3 SLAM multi-capteurs

La fusion de plusieurs modalités permet de combiner leurs forces respectives. Les travaux récents s’appuient sur des graphes de facteurs (GTSAM, g2o) pour intégrer des contraintes hétérogènes (LiDAR, caméra, IMU, GPS).

2.4 SLAM thermique (LWIR)

Le SLAM basé sur caméra [LWIR](#) est un domaine émergent. Les images thermiques sont invariantes aux conditions d'éclairage visible, ce qui les rend attractives pour les environnements obscurs ou enfumés. Cependant, leur faible résolution et le manque de texture fine posent des défis pour l'extraction de *keypoints*. Des avancées récentes comme le Thermal-SLAM qui réduit l'impact du bruit thermique et améliore la détection de *features*[\[3\]](#).

2.5 3D Gaussian Splatting

Introduit par Kerbl *et al.* (2023), le [3D Gaussian Splatting \(3DGS\)](#) représente une scène 3D par un ensemble de gaussiennes 3D anisotropes, permettant un rendu en temps réel de haute qualité. Son intégration dans un pipeline SLAM (*SplaTAM*, *MonoGS*) ouvre la voie à des cartes denses et photo-réalistes.[\[4\]](#)

Chapitre 3

User Stories

Les *user stories* suivantes ont été définies par mes soins pour permettre une meilleure compréhension de l'objectif principal et afin de créer un Kanban.

Table 3.1: User stories principales

ID	Rôle	Objectif	Priorité
US-01	Developpeur	Comprendre l'architecture matérielle et logicielle du TurtleBot 4	Haute
US-02	Developpeur	Calibrer la caméra RGB et le LiDAR (intrinsèque + extrinsèque)	Haute
US-03	Developpeur	Lancer un pipeline SLAM RGB et visualiser la carte en temps réel dans RViz2	Haute
US-04	Developpeur	Lancer un pipeline SLAM LiDAR et comparer la précision avec US-03	Haute
US-05	Developpeur	Fusionner les données RGB + LiDAR pour améliorer la robustesse du SLAM	Moyenne

Chapitre 4

Spécification des besoins

4.1 Besoins fonctionnels

- BF-1** Le système doit permettre le pilotage du TurtleBot-4 via une téléopération [ROS 2](#) clavier.
- BF-2** Le système doit acquérir et synchroniser les flux de données de la caméra RGB-D (OAK-D Pro) et du LiDAR (RPLiDAR A1M8).
- BF-3** Le système doit réaliser la calibration intrinsèque et extrinsèque de la caméra et du LiDAR.
- BF-4** Le système doit exécuter un algorithme de SLAM RGB en temps réel et publier la carte et la pose estimée sur les topics ROS 2 adéquats.
- BF-5** Le système doit exécuter un algorithme de SLAM LiDAR en temps réel.
- BF-6 (Bonus)** Le système doit supporter l'acquisition de données LWIR et exécuter un pipeline SLAM thermique.
- BF-7 (Bonus)** Le système doit intégrer un module de 3D Gaussian Splatting alimenté par les poses estimées par le SLAM.

4.2 Besoins non fonctionnels

- BNF-1 Performance** : le pipeline SLAM doit pouvoir lancer soit le SLAM RGB, soit le SLAM LiDAR en temps réel (fréquence d'au moins 5 Hz) sur le User PC.
- BNF-2 Portabilité** : le code doit être compatible Ubuntu 22.04 / ROS 2 Jazzy.

4.3 Tests

Table 4.1: Plan de tests

ID	BF associé	Scénario	Critère de succès
T-01	BF-1	Téléopérer le robot sur 5 m en ligne droite et avoir des retours camera stable sur rviz	Déviaton < 5 cm
T-02	BF-3	Calibration caméra sur davier	Erreur de reprojection < 0.4 px
T-03	BF-4	SLAM RGB sur séquence corridor 3 m	ATE < 0.1 m
T-04	BF-5	SLAM LiDAR sur même séquence	ATE < 0.05 m
T-05	BF-6	Comparaison RGB vs LiDAR	Rapport d'évaluation généré

Chapitre 5

Choix logiciels

Ce chapitre présente l'ensemble des outils, bibliothèques et langages retenus pour le développement. Les choix ont été fait en fonction de s'il etait compatible avec ROS 2 Jazzy et s'ils etaient facilement integrable dans le projet.

5.1 Système d'exploitation et middleware

Le systeme d'exploitation ici ce fait de soit car pour assurer la bonne continuite du projet et pour eviter les problemes de compatibilité il faut choisir un systeme d'exploitation compatible avec ROS 2 Jazzy.

Ubuntu 22.04 LTS Distribution de référence pour ROS 2 Jazzy, bénéficiant d'un support long terme.

ROS 2 Jazzy Version LTS de ROS 2 offrant une architecture DDS fiable, des outils de débogage matures (RViz2, rqt, ros2bag) et une compatibilité native avec TurtleBot 4.

5.2 Algorithmes de SLAM

Le SLAM a ete implemente "from scratch" pour le LiDAR (ICP) et la calibration extrinsèque, afin de mieux comprendre les algorithmes de base.

ICP Implémenté à partir de zéro pour l'odométrie LiDAR, en utilisant une approche de correspondance de points et d'estimation de transformation rigide.

Bresnham Utilisé pour trouver les correspondances entre les points du LiDAR et les pixels.

ORB-SLAM3 Utilisé pour la branche SLAM visuel pur (RGB) afin de disposer d'une référence académique reconnue.

5.3 Calibration

La calibration de la camera n'as pas ete l'objectif principal du projet alors le choix d'utiliser une solution existante a été fait pour gagner du temps et se concentrer sur le SLAM.

camera_calibration (ROS 2) Calibration intrinsèque caméra via damier (modèle pinhole + Brown-Conrady).

5.4 Évaluation

L'évaluation de la précision de l'odométrie LiDAR est réalisée à l'aide de la bibliothèque `rpg_trajectory_evaluation` développée par l'Université de Zurich, qui fournit des métriques standardisées (ATE, RPE).

rpg_trajectory_evaluation Pour l'évaluation de la précision de l'odométrie LiDAR via https://github.com/uzh-rpg/rpg_trajectory_evaluation

5.5 Développement et versioning

C++ Langage de programmation compatible avec ROS 2 Jazzy.

Git + GitHub Versioning et collaboration.

Gazebo Simulateur de robotique 3D pour les tests préliminaires en simulation avant déploiement sur le robot réel. De plus le turtlebot 4 a une simulation officielle disponible.

Colcon Outil de build recommandé pour ROS 2, facilitant la gestion des dépendances et l'organisation du workspace.

Python 3.10 et OpenCV Langage de programmation utilise pour la pipeline caméra avec OpenCV.

C++ Langage de programmation pour le LIDAR ICP, offrant de meilleures performances.

LLMs (Claude, Gemini) Utilisés pour la génération de code, la rédaction de documentation et l'assistance à la résolution de problèmes.

Chapitre 6

Architecture

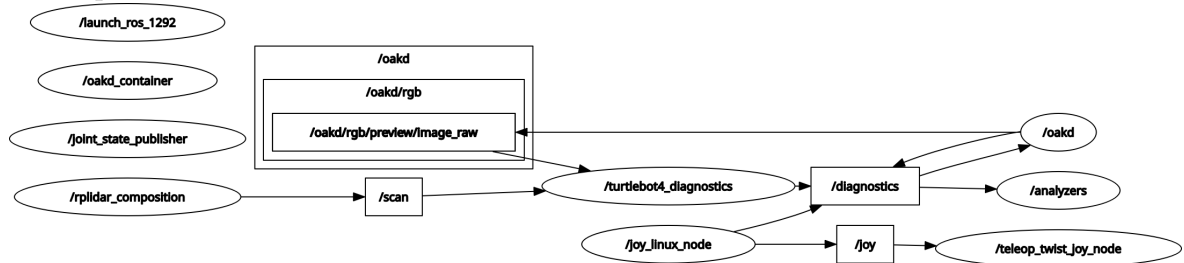
6.1 Architecture matérielle

La plateforme TurtleBot 4 est composée des éléments matériels suivants:

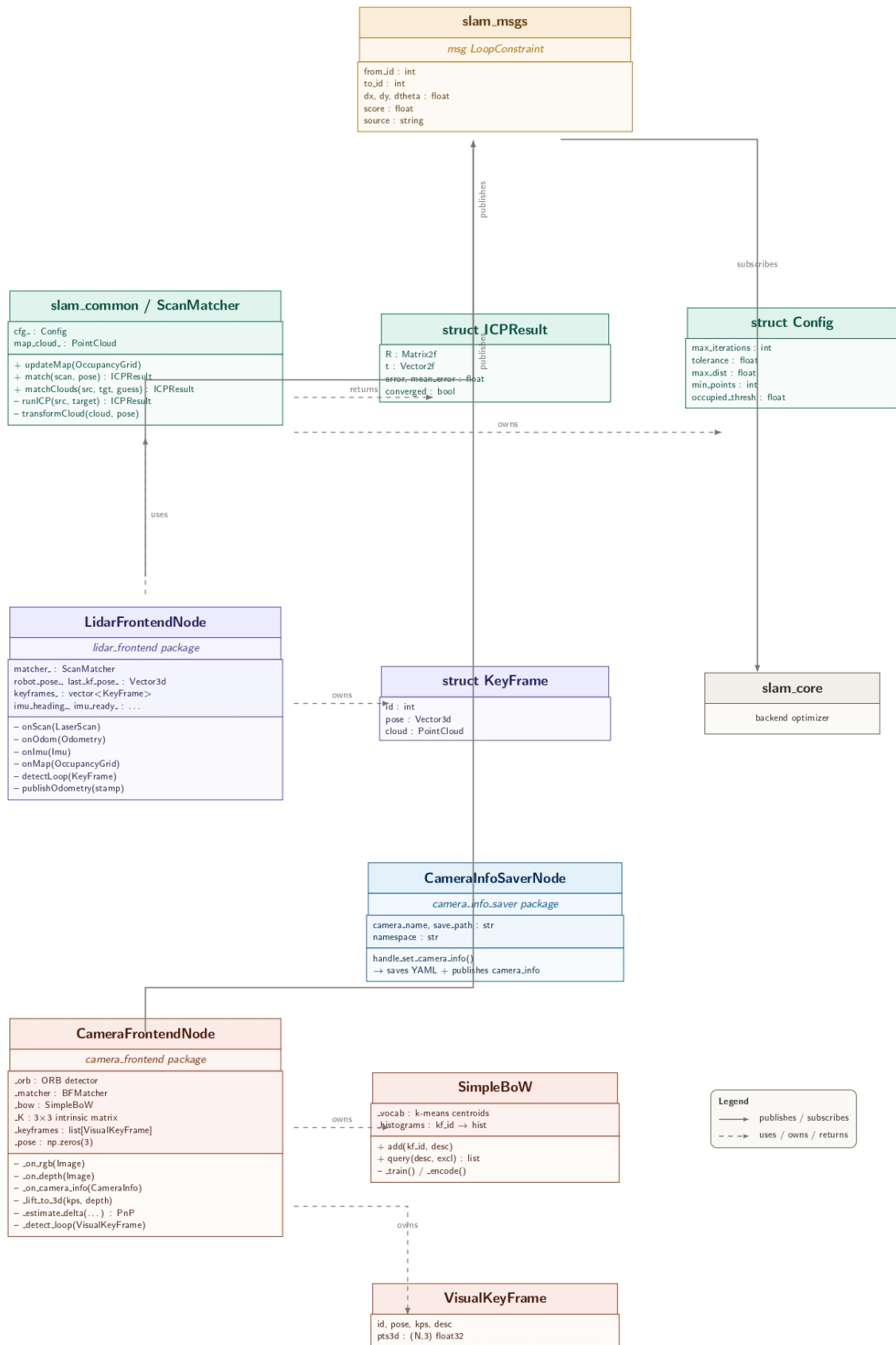
- **Base iRobot Create 3**: plateforme de déplacement différentiel avec encodeurs et IMU intégrés.
- **Raspberry Pi 4B (4 Go)**: calculateur embarqué principal.
- **Caméra OAK-D Pro**: caméra RGB-D stéréo avec accélérateur neural intégré (Intel MyriadX).
- **LiDAR RPLiDAR A1M8**: scanner laser 2D 360° à 12 Hz.

6.2 Architecture logicielle ROS 2

L'architecture logicielle du projet est organisée en plusieurs nœuds ROS 2 communicants via des topics et des services:



6.3 Diagramme de cas d'utilisation (UML)



ROS 2 Topics & Services

```
Lidar frontend  sub: /scan /odom /imu /map  pub: /lidar/odometry /lidar/loop_constraint
```

Camera frontend sub: /oakd/rgb/image_raw • /oakd/stereo/image_raw • /oakd/rgb/camera_info pub: /camera/odometry • /camera/loop_constraint

Camera info saver srv: set_camera_info → saves YAML + republishes camera_info

slam_core sub: /lidar/loop_constraint + /camera/loop_constraint → pose-graph optimisation

Chapitre 7

Réalisation

7.1 Mise en place de l'environnement

L'environnement de développement a été configuré comme suit:

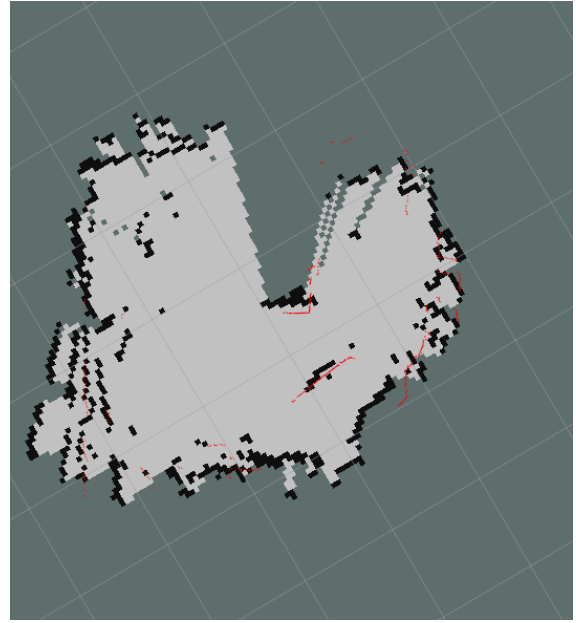
```
1 sudo apt update && sudo apt install ros-Jazzy-desktop -y
2
3 sudo apt install ros-Jazzy-turtlebot4-desktop -y
4
5 git clone https://github.com/xdZireael/PFE
6
7 cd PFE_ws && colcon build --symlink-install
8
9 source install/setup.bash
```

Listing 7.1: Installation de l'environnement ROS 2 et TurtleBot 4

7.2 Screenshots



(a) Carte SLAM benchmark dans RViz2



(b) Carte SLAM LiDAR dans RViz2

Figure 7.1: Visualisation des cartes générées par les deux pipelines SLAM

7.3 Algorithmes et méthodes

7.3.1 ICP from scratch

Le code de l'algorithme ICP est le suivant:

Algorithm 1 ICP (Iterative Closest Point) pour l'odométrie LiDAR

Require: $P = \{p_i\}$, $Q = \{q_j\}$ (nuages de points source et cible)

Ensure: Transformation rigide $T \in SE(3)$ alignant P sur Q

- 1: Initialiser $T \leftarrow I$ (identité)
- 2: **repeat**
- 3: Trouver les correspondances $C = \{(p_i, q_j)\}$
- 4: Estimer la transformation T minimisant l'erreur de correspondance:

$$T = \arg \min_T \sum_{(p_i, q_j) \in C} \|Tp_i - q_j\|^2$$

- 5: Appliquer T à P : $P \leftarrow TP$
 - 6: **until** convergence (changement de T inférieur à un seuil) ou nombre maximal d'itérations atteint
-

Algorithm 2 Détection de fermeture de boucle LiDAR avec une pipeline à trois verrous

Require: Image courante de keyframe $\mathbf{k}_{\text{cur}}$ avec sa pose $\mathbf{p}_{\text{cur}}$ et son nuage de points $\mathcal{P}_{\text{cur}}$, historique de keyframes $\{\mathbf{k}_i\}_{i=0}^{n-1}$, rayon de recherche spatiale $r_{\text{lc}} = 1.0$ m, âge minimal de séparation $A = 5$ keyframes pour éviter les faux positifs immédiats, seuil de qualité ICP $\tau_{\text{icp}} = 0.10$ m, distance maximale d’inlier $d_{\text{in}} = 0.10$ m, ratio minimal d’inliers $\rho_{\text{min}} = 0.65$

Ensure: Message de contrainte de boucle, ou absence de boucle détectée

- 1: **for** chaque keyframe candidate $\mathbf{k}_i$ avec $i < n_{\text{cur}} - A$ **do**
- 2: **Verrou 1 proximité spatiale.** Calculer la distance euclidienne entre la position courante et la position de la candidate. Rejeter immédiatement si la distance dépasse le rayon de recherche r_{lc} , ce qui évite tout recalage ICP coûteux sur des paires éloignées.
- 3: **Verrou 2 recalage ICP entre les deux nuages.** Construire une pose relative initiale à partir des poses stockées des deux keyframes, puis lancer l’ICP (Algorithme 1) entre $\mathcal{P}_{\text{cur}}$ et $\mathcal{P}_i$. Rejeter la candidate si l’ICP n’a pas convergé ou si l’erreur résiduelle dépasse τ_{icp} :

$$(\mathbf{R}, \mathbf{t}, \bar{e}, \text{conv}) \leftarrow \text{ICP}(\mathcal{P}_{\text{cur}}, \mathcal{P}_i, \Delta \mathbf{p})$$

- 4: **Verrou 3 test du ratio d’inliers.** Appliquer la transformation ICP à chaque point du nuage courant, puis compter les points dont le voisin le plus proche dans $\mathcal{P}_i$ est à moins de d_{in} . Ce test détecte les convergences ICP spurieuses vers un minimum local géométriquement plausible mais physiquement incorrect:

$$\rho \leftarrow \frac{\#\{\mathbf{s} \in \mathcal{P}_{\text{cur}} \mid \min_j \|\mathbf{R}\mathbf{s} + \mathbf{t} - \mathbf{t}_j\| < d_{\text{in}}\}}{|\mathcal{P}_{\text{cur}}|}$$

Rejeter si $\rho < \rho_{\text{min}}$.

- 5: **Les trois verrous sont passés.** Émettre le message de contrainte de boucle avec l’identifiant de la keyframe source, l’identifiant de la keyframe cible, le déplacement relatif $(\mathbf{t}, \Delta \theta)$ déduit de $\mathbf{R}$, et le score de qualité $\bar{e}$. Retourner immédiatement (une seule boucle par keyframe).
 - 6: **end for**
 - 7: Aucune fermeture de boucle détectée pour cette keyframe
-

Algorithm 3 EKF — étape de correction par mesure de pose (LiDAR ou caméra)

Require: Moyenne prédite $\boldsymbol{\mu}^-$ et covariance prédite $\boldsymbol{\Sigma}^-$, mesure de pose $\mathbf{z} = (x_z, y_z, \theta_z)^\top$ issue du frontend LiDAR ou caméra, matrice de bruit de mesure $\mathbf{R}$ propre au capteur (LiDAR: $\sigma_{xy} = 0.03$ m, $\sigma_\theta = 0.02$ rad; caméra: $\sigma_{xy} = 0.05$ m, $\sigma_\theta = 0.03$ rad)

Ensure: Moyenne corrigée $\boldsymbol{\mu}$ et covariance corrigée $\boldsymbol{\Sigma}$

- 1: Calculer la matrice d'innovation $\mathbf{S}$, somme de la covariance prédite et du bruit de mesure. Le modèle d'observation étant l'identité ($\mathbf{H} = \mathbf{I}_3$), il n'y a pas de projection:

$$\mathbf{S} \leftarrow \boldsymbol{\Sigma}^- + \mathbf{R}$$

- 2: Calculer le gain de Kalman $\mathbf{K}$, qui pondère la confiance accordée à la mesure par rapport à la prédiction:

$$\mathbf{K} \leftarrow \boldsymbol{\Sigma}^- \mathbf{S}^{-1}$$

- 3: Calculer le vecteur d'innovation $\boldsymbol{\nu}$, écart entre la mesure et la prédiction. La composante angulaire est ramenée dans $(-\pi, \pi]$ pour gérer les discontinuités à $\pm\pi$:

$$\boldsymbol{\nu} \leftarrow \mathbf{z} - \boldsymbol{\mu}^-, \quad \nu_\theta \leftarrow \text{wrapToPi}(\nu_\theta)$$

- 4: Corriger la moyenne en appliquant le gain au vecteur d'innovation, puis normaliser l'angle résultant:

$$\boldsymbol{\mu} \leftarrow \boldsymbol{\mu}^- + \mathbf{K} \boldsymbol{\nu}, \quad \theta \leftarrow \text{wrapToPi}(\theta)$$

- 5: Réduire la covariance pour refléter le gain d'information apporté par la mesure:

$$\boldsymbol{\Sigma} \leftarrow (\mathbf{I}_3 - \mathbf{K}) \boldsymbol{\Sigma}^-$$

- 6: **return** la moyenne corrigée $\boldsymbol{\mu}$ et la covariance corrigée $\boldsymbol{\Sigma}$
-

7.3.2 Calibration extrinsèque caméra/LiDAR

La calibration extrinsèque consiste à estimer la transformation rigide ${}^L\mathbf{T}_C \in SE(3)$ entre le repère LiDAR $\mathcal{F}_L$ et le repère caméra $\mathcal{F}_C$:

$${}^L\mathbf{T}_C = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3) \quad (7.1)$$

Chapitre 8

Résultats et tests

8.1 Résultats expérimentaux

Table 8.1: Comparaison des performances SLAM sur la séquence de test

Méthode	ATE RMSE (m)	RPE RMSE (m)	Fréquence (Hz)	Ferm
ORB-SLAM3 (RGB)	—	—	—	
KISS-ICP (LiDAR)	—	—	—	
RTAB-Map (RGB+LiDAR)	—	—	—	
RTAB-Map (LWIR)	—	—	—	

Chapitre 9

Conclusion

9.1 Bilan

Ce projet a permis d'améliorer la prise en main de la plateforme TurtleBot 4 ainsi que le fonctionnement des pipelines SLAM en général. La plateforme TurtleBot 4 s'est avérée simple à prendre en main, hors mis la mise en place de l'environnement ROS. Le pipeline MC-SLAM développé sous ROS 2 montre la difficulté de l'adaptation sur les séquences testées. Au final, je ne suis pas content du résultat final, le projet a été plus difficile que prévu et je n'ai pas réussi à faire fonctionner les pipelines SLAM de manière satisfaisante, que ce soit en RGB ou en LiDAR. Par contre, j'ai appris énormément sur les algorithmes de SLAM, la calibration multi-capteurs et les défis de l'implémentation sur une plateforme embarquée.

9.2 Perspectives

- **Temps réel embarqué** : optimiser les algorithmes pour les contraintes du Raspberry Pi 4 (TensorRT, ONNX Runtime).
- **SLAM sémantique** : intégrer la reconnaissance d'objets pour une cartographie sémantique.
- **Navigation autonome** : coupler le SLAM à un planificateur de chemin (Nav2) pour l'exploration autonome.
- **3D Gaussian Splatting** : finaliser l'intégration et évaluer la qualité de rendu sur les cartes construites.

Références bibliographiques

- [1] C. CAMPOS, R. ELVIRA, J. J. G. RODRÍGUEZ, J. M. M. MONTIEL et J. D. TARDÓS, « ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM, » *IEEE Transactions on Robotics*, t. 37, n° 6, p. 1874-1890, 2021.
- [2] I. VIZZO, T. GUADAGNINO, B. MERSCH, L. WIESMANN, J. BEHLEY et C. STACHNISS, « KISS-ICP: In Defense of Point-to-Point ICP — Simple, Accurate, and Robust Registration If Done the Right Way, » *IEEE Robotics and Automation Letters*, 2023.
- [3] Y. WU, L. WANG, L. ZHANG et al., « Improving Autonomous Detection in Dynamic Environments with Robust Monocular Thermal SLAM System, » *ISPRS Journal of Photogrammetry and Remote Sensing*, t. 203, p. 265-284, 2023, ISSN : 0924-2716. DOI : [10.1016/j.isprsjprs.2023.08.002](https://doi.org/10.1016/j.isprsjprs.2023.08.002).
- [4] B. KERBL, G. KOPANAS, T. LEIMKÜHLER et G. DRETTAKIS, « 3D Gaussian Splatting for Real-Time Radiance Field Rendering, » *ACM Transactions on Graphics*, t. 42, n° 4, 2023.